Ball Trajectory Prediction

P76144671 Liang-Ying, Huang
NCKU

Abstract

In modern sports analytics, accurately predicting the future trajectory of a ball is a critical but challenging task due to the high speed, motion blur, and small size of the object. This report investigates three distinct deep learning architectures for ball trajectory prediction using data derived from the TrackNet dataset. The models observe the first 10 frames of a sequence and predict the ball's spatial coordinates in the subsequent 10 frames. We evaluate a trajectory-only model based on a Bidirectional LSTM, a vision-only model utilizing a pre-trained ResNet18 feature extractor paired with sequential LSTM modeling, and a multi-modal fusion network combining visual feature maps and spatial embeddings. Our experiments detail crucial data preprocessing steps—including coordinate normalization, aspect-ratio-preserving image resizing, and group-based data splitting—alongside rigorous training configurations utilizing Smooth L1 Loss and heavy augmentations. The models are evaluated using Average Displacement Error (ADE) and Final Displacement Error (FDE) in pixel space.

1. Introduction

Predicting the future trajectory of a ball is a foundational problem in sports analytics. Systems that can reliably anticipate ball movement are essential for understanding game dynamics, analyzing player behavior, and supporting advanced officiating technologies. However, this task presents unique computer vision and sequence modeling challenges, primarily because the ball often occupies only a fraction of a pixel space, moves at high velocities, and suffers from motion blur and occlusion.

This work explores the efficacy of different data modalities for trajectory prediction, formulating the problem as a sequence-to-sequence prediction task: given 10 observed frames, the model must output the (x, y) coordinates for the next 10 frames. To comprehensively study this, this report implements and compares three approaches:

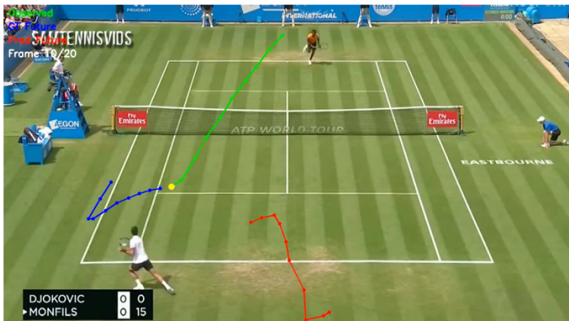

Figure 1. Example of ball trajectory prediction. The model observes the first 10 frames and predicts the ball positions in the next 10 frames. A typical visualization may show the observed trajectory in green, the ground-truth future trajectory in blue, and the predicted future trajectory in red.

1. **Trajectory-only Model:** Relies exclusively on past (x, y) coordinate sequences.
2. **Vision-only Model:** Relies entirely on raw RGB image sequences.
3. **Fusion Model:** Combines both visual data and trajectory coordinates to leverage spatial context alongside explicit motion history.

By evaluating these models using Average Displacement Error (ADE) and Final Displacement Error (FDE), this report aims to identify the most robust representations for tracking and predicting tiny objects.

2. Methodology

These architectures process input sequences to predict future spatial coordinates. We define the input sequence as the history of the image frames, coordinates, or both, from time step 0 to 9, and the target as the coordinates (x, y) from time step 10 to 19.

2.1. Trajectory-only Model

The trajectory model utilizes a sequence of 2D coordinates. Because momentum and direction are heavily dependent on both past and future contextual dependencies within the 10-

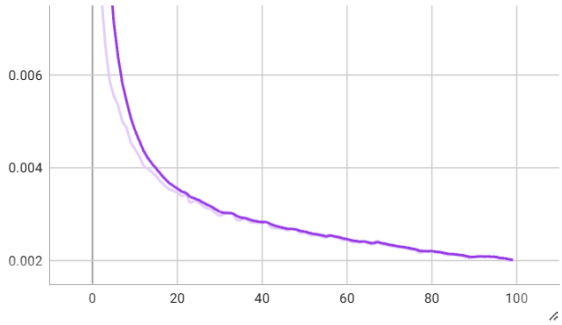

Figure 2. Training loss of trajectory-only model.

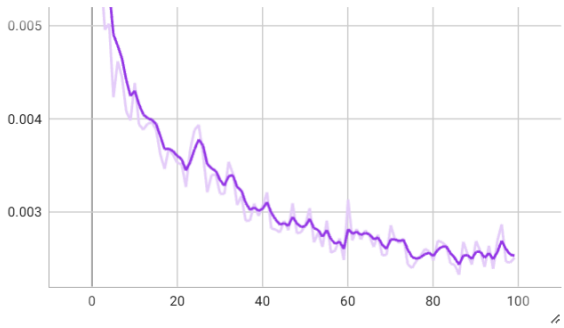

Figure 3. Validation loss of trajectory-only model.

frame window, we employ a Bidirectional Long Short-Term Memory (Bi-LSTM) network. The coordinate sequence is fed into a 2-layer Bi-LSTM with 128 hidden units. The final hidden state of the sequence is passed through a Multi-Layer Perceptron (MLP) consisting of a linear layer, ReLU activation, dropout, and a final linear layer to output the predicted 10×2 coordinate matrix.

2.2. Vision-only Model

The vision model extracts spatial features directly from RGB frames. We utilize a ResNet18 backbone, pre-trained on ImageNet, as our feature extractor. The final classification layers are removed, and the extracted feature maps (flattened to 512 dimensions) for each of the 10 input frames are passed sequentially into a 2-layer LSTM. The final hidden state is processed by an MLP to output the coordinate predictions.

2.3. Fusion Model

The fusion model aims to leverage the physical precision of the trajectory data while utilizing the environmental context from the vision data.

- **Vision Pathway:** Image frames are processed through a custom Convolutional Neural Network (CNN) featuring 3×3 convolutions, max pooling, and adaptive average pooling to extract a 64-dimensional feature vector per frame.
- **Trajectory Pathway:** The (x, y) coordinates are embedded into a 32-dimensional vector per frame using a linear layer.
- **Fusion:** The visual and trajectory features are concatenated along the feature dimension to form a 96-dimensional combined vector for each time step. This fused sequence is fed into an LSTM, followed by an MLP for the final prediction.

3. Experiments and Discussion

3.1. Experimental Settings

All models were implemented in PyTorch and optimized using Adam. The trajectory and fusion models were trained using Mean Squared Error (MSE) loss with a learning rate of 0.001 and batch sizes of 64 and 8, respectively.
Due to the inherent difficulty of extracting coordinates from images, the vision-only model utilized a Smooth L1 Loss ($\beta = 0.02$) to gracefully handle spatial outliers. It was trained with a batch size of 8, weight decay ($1e{-}4$), and gradient clipping (max norm of 1.0) to prevent exploding gradients. Furthermore, we employed a ReduceLROnPlateau scheduler (factor of 0.5, patience of 3) and an early stopping mechanism (patience of 10) to combat overfitting.

3.2. Data Preprocessing and Augmentation

The dataset contains games divided into clips, where "Clip1" of each game is strictly reserved for testing. We split the trajectory into windows of 20 frames. To maintain the physical geometry of the trajectories, images were resized to 288×512, preserving the original 16:9 aspect ratio of the 1280×720 source videos. All ground truth (x, y) coordinates were normalized to a $[0, 1]$ scale to stabilize network gradients. To prevent data leakage, we implemented a group-based splitting algorithm ensuring that overlapping windows from the same video clip were strictly isolated to either the training or validation set. For the vision and fusion models, rigorous data augmentation was applied during training, including ColorJitter, RandomAffine (translation and scaling), and RandomErasing, forcing the networks to learn robust representations of the tiny objects despite occlusions and lighting variations.

3.3. Metrics and Results

Models were evaluated using ADE and FDE. If the ground-truth ball was invisible in a frame, it was explicitly skipped during metric computation. The metrics are defined as:

$$ADE = \frac{1}{T} \sum_{t=1}^{T} ||\hat{p}_t - p_t||$$

$$FDE = ||\hat{p}_T - p_T||$$

where $\hat{p}$ is the predicted position and p is the ground-truth position in pixels.

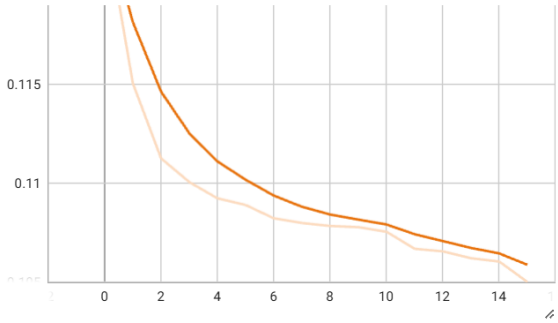

Figure 4. Training loss of vision-only model.

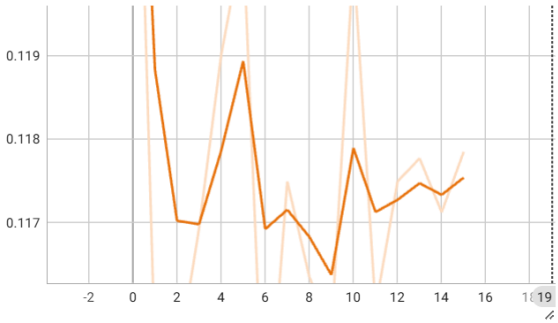

Figure 5. Validation loss of vision-only model.

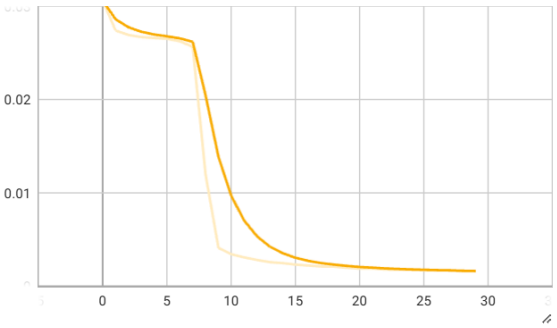

Figure 6. Training loss of fusion model.

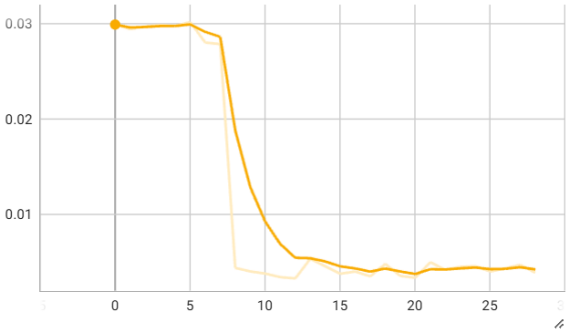

Figure 7. Validation loss of fusion model.

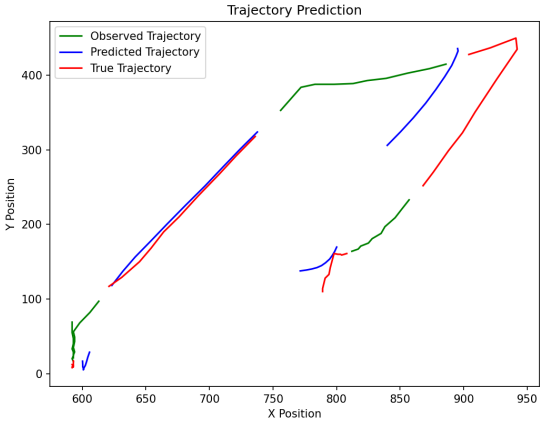

Figure 8. An example of predicted trajectories.

3.4. Challenges, Limitations, and Observations

- **Stable Convergence of Trajectory Baseline:** As illustrated in the loss curves, the trajectory-only model exhibited stable and consistent convergence over 100 epochs. The validation loss tracked the training loss closely without significant overfitting, confirming that sequential coordinate data provides a highly reliable signal for predicting future momentum and positioning.
- **Tiny Object Tracking and Vision Volatility:** The vision-only model proved to be the most difficult to train. Compressing a high-definition image to a 288×512 resolution shrinks the ball to a fraction of a pixel. Despite utilizing Smooth L1 loss and learning rate scheduling, the model struggled to generalize without explicit motion cues (e.g., optical flow), highlighting the limitations of standard CNN backbones for tiny object tracking.
- **Modality Collapse in Fusion:** The fusion model's loss curves reveal a clear two-phase convergence. For the first 7 epochs, the loss plateaued as the model attempted to balance the noisy visual features with the spatial coordinates. At epoch 8, the loss experienced a sharp, sudden drop and smoothly converged thereafter. This "cliff" strongly suggests the occurrence of modality collapse. Because the (x, y) trajectory coordinates provide a dense, easily parsable signal compared to the complex vision features, the LSTM likely learned to heavily prioritize the trajectory pathway and effectively ignore the visual data. Consequently, simple feature concatenation did not yield a synergistic improvement over the trajectory-only baseline.

4. Conclusion

In this homework, we successfully implemented and evaluated three modalities for ball trajectory prediction. The trajectory-only model provided a strong, lightweight base-

	G1	G2	G3	G4	G5	G6	G7	G8	G9	G10	Avg
Trajectory ADE	21.38	46.68	27.01	36.01	28.14	42.33	53.52	35.48	19.93	29.98	34.05
Trajectory FDE	38.95	70.55	54.48	62.12	50.14	80.26	85.99	59.78	29.32	49.41	58.10
Vision ADE	172.11	211.05	165.93	137.75	127.65	159.17	179.06	160.67	204.32	146.52	166.42
Vision FDE	195.63	204.00	174.54	137.48	125.06	164.55	227.32	175.23	198.54	143.95	174.63
Fusion ADE	30.71	63.53	40.23	47.29	36.79	57.16	64.39	47.46	44.83	42.89	47.53
Fusion FDE	60.02	94.18	63.92	76.72	55.02	93.17	104.08	82.09	60.92	66.43	75.65

Table 1. ADE/FDE results for each of the 10 games separately, and the average ADE/FDE results.

line by directly leveraging sequential momentum. While the vision-only model struggled due to the tiny spatial footprint of the object and the lack of explicit motion features (like optical flow), the fusion model demonstrated the architectural potential of multi-modal networks. Future improvements should explore cross-attention mechanisms to explicitly guide the visual network using trajectory queries, and the use of frame-differencing to highlight moving objects before feature extraction.